Compile Once, Reuse Safely

Data Certificates and Selective Deoptimization for Preprocessing Pipelines

Nikhil V
School of Computer Science and Engineering, Vellore Institute of Technology
Vellore, India
atnikhil06@gmail.com

Abstract

Batch data-preparation pipelines are compiled once and run again on every new batch: next month's file has the same layout and different values. A compiler that reads the input data at compile time can prove a pipeline's contracts and delete the checks and operations the data makes useless, but its proofs are about one file. We present PrepLang, a compiler for table and image preprocessing pipelines that computes an abstract dataset state from the data, or from its metadata alone, decides every contract as proven, violated or deferred, and optimizes, together with three ways to carry its proofs to new batches. *Data certificates*, found by re-analysing the program under counterfactually weakened input facts, are sets of input facts that the proofs and removals depend on, 1-minimal for programs without derived columns. *Selective deoptimization* keeps every removed check and operation in the generated program with its own item certificate: a new batch runs the optimized program, and only the removals whose certificates it breaks run again; a batch is refused only for facts no run-time code can stand in for, such as a column's type. We prove this exact for every admitted batch and mechanize the proof in Lean 4, end to end for a core of the language. *Composition* lets a compiled stage publish a guarantee for each table it writes; checking it against the next stage's certificates shows which downstream checks can never fire and writes the upstream contracts that would close the gap. On 238 real monthly batches of three public datasets, selective deoptimization handles 221 batches exactly as a fresh compilation would and none unsoundly, where Great Expectations and pandera, set up on the same data, refuse 92–93% of the months a fresh compilation runs, and TFDV, which checks no numeric ranges, admits months on which the optimized program does not do what a fresh compilation does. Pandas scripts from GitHub, lifted into PrepLang, reproduce their own results on new batches: 3198 of 3212 runs of 292 held-out scripts for two Kaggle tasks, and all 3255 monthly runs of 72 scripts written by others for the traffic and bike data; none differ. Because selective deoptimization never relies on the compile-time facts being true, PrepLang can also compile against data the analyst may not see: from a differentially private histogram of each column over the program's own numbers, everything the compiler writes is ε-differentially private, and 38,409 runs of 2,919 private compilations, from noisy, absent or random facts, were all exact.

Keywords

data-aware compilation, abstract interpretation, data validation, speculative optimization, deoptimization, data contracts, machine learning pipelines

1 Introduction

Data preparation is a large part of building machine-learning systems, and its errors propagate silently into models [1, 2]. A preparation pipeline is usually a Python script built on pandas [3] and OpenCV [4]. Such a script cannot know, until it runs, whether the column it normalizes holds numbers, whether a folder contains a grayscale image among color ones, or whether a filter will keep any row. When an assumption is wrong, the script raises an exception after processing part of the data or, worse, finishes and writes a wrong dataset.

Batch preparation has a property most program analyses leave unused: *the inputs exist when the pipeline is compiled.* PrepLang, a small language and compiler for table and image preprocessing, makes the data a compile-time input, as F# type providers do for types [18]. Its analysis reads every CSV cell, or only a Parquet footer or the image headers, and follows the types, missing values, value ranges and image properties through the pipeline. Contracts the state decides are proven or reported as violated, with the offending row, before any data is processed; the optimizer deletes the checks of proven contracts and the operations the state shows to be useless.

That raises the question this paper answers. A pipeline compiled and proven for this month's file runs again next month, on a file the proofs were not made for. Recompiling every batch is possible, but it gives up three things: the compiled program as a fixed, reviewed and tested artifact whose manifest records the proofs it relies on; the signal that the data has drifted away from what the program was optimized for, since a fresh compilation silently produces a different program; and any static guarantee between the stages of a pipeline, since every stage would change with every batch. A fingerprint of the compiled file keeps all three but refuses every new batch. Data-validation tools [5–8] infer expectations from the data rather than from the program: on our real monthly data, the suites they infer refuse most new months, and the one that leaves numeric ranges out admits months on which the optimized program does not do what a fresh compilation does (Section 10). What is missing is the program's side of the question: *which facts of the compile-time data does the compiled program depend on, and what can it do for a batch that lacks one?*

Contributions.

- **Data-aware compilation** for tables, dates, derived columns and images (Section 3): one abstract state, computed from the data or from metadata only, used for type checking, for hybrid checking of value contracts [19], and for optimization.
- **Data certificates** (Section 4): by analysing the program again under counterfactually weakened input facts, the

compiler finds a set of input facts that its proofs and removals depend on (1-minimal without derived columns), over every column of a file, the dates of a column, and boxes of image shapes, using the forward analysis as a black box.

- **Selective deoptimization** (Section 5): the optimizer removes instructions *speculatively*; the compiler derives an admission certificate (the facts no run-time code can stand in for) and an item certificate for every removal. On a new batch the generated program re-enables exactly the removals whose item certificates the batch breaks. We prove that this computes exactly what the source program computes on every admitted batch, and mechanize the proof in Lean 4 [60]: for any monotone, sound analysis, and end to end, with the analysis itself, for a core of PrepLang.
- **Composition** (Section 6): every compiled stage publishes a guarantee for each table it writes. An assume-guarantee check against the next stage's certificates shows which of its checks can never fire, and synthesizes the contract a producer must meet so that none of its consumers ever refuses or deoptimizes.
- **Private compilation** (Section 7): an analyst can compile against data they may not see. No differentially private release can certify the emptiness facts a data-aware compiler optimizes with, so optimizing from released facts needs run-time guards, which selective deoptimization provides. The data owner releases one Laplace histogram per column over the cells the program's own numbers define; everything the compiler writes is ε-differentially private for a declared schema, and with probability $1 - \beta$ every "likely violated" warning is true and every released fact is false by few rows. At equal budget these program-chosen cells keep more of the normal compilation's removals than fixed bins, report broken contracts more often at a small budget, and give a program where synthetic data compiled normally reports contracts as violated that the data satisfies (Section 10.8).
- **An evaluation on real data and real code** (Section 10): 238 monthly batches of three public datasets, scored against fresh compilations and against pandera [7], Great Expectations [8] and TFDV [5]; 292 held-out pandas scripts for two Kaggle tasks, lifted into PrepLang by a second front end, and 72 scripts by others run on the real months; two-stage pipelines; private compilations from noisy, absent and random facts; injected mistakes and 44 real failures from GitHub issues; and the cost of all of it.

Each ingredient has a precedent: speculation with a fallback path in data compilers [49, 50], deoptimization [34, 51], validity ranges of query plans [35], residual run-time checks [53], abductive and sufficient conditions [47, 48], inferred input specifications [43], and data validation. What is new, to our knowledge, is to derive per-removal input conditions from the proofs of a data-aware compiler, to check them once per batch on cheap statistics, to re-enable exactly the broken removals with an exactness theorem, and to compose the resulting certificates across pipeline stages (Section 11).

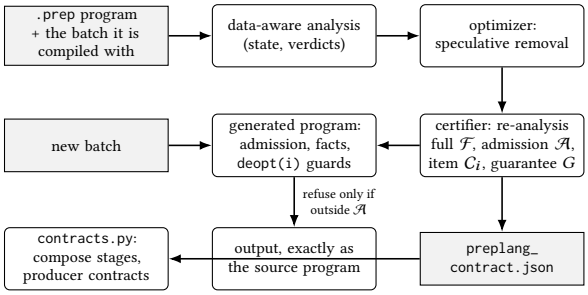


Figure 1: How PrepLang compiles once and reuses the result. The certifier re-runs the analysis with weaker input facts, reading no data; on a new batch, the generated program runs a removed instruction i only if the batch breaks its item certificate C_i.

```
1 LOAD "students.csv" AS s;
2 EXPECT s.marks BETWEEN 0 AND 100;
3 EXPECT s.name NOT_NULL;
4 FILL_NULL s.marks WITH 0;
5 FILTER s WHERE marks >= 10;
6 SAVE s AS "out.csv";
```

Figure 2: A pipeline whose four statements on lines 2–5 the optimizer removes.

2 Overview

Fig. 2 cleans a table of students. Compiled against a file whose marks lie in $[35, 95]$ and whose names are all present, PrepLang proves the contracts on lines 2–3, finds that `FILL_NULL` on line 4 and `FILTER` on line 5 change nothing, and generates a program without any of the four.

Certificates. The certifier analyses the program again with weaker facts: marks may be missing, marks have no lower bound, and so on. A weakening is kept if the proofs survive. The *full certificate* keeps all four removals: marks in $[10, 100]$ without missing values, names without missing values, and the types of every column. Any batch inside it can run the optimized program unchanged.

Deoptimization. A batch with a mark of 101 is outside the full certificate. Without deoptimization it must be refused. With it, the compiler has also found an *item certificate* per removal: the check on line 2 needs marks in $[0, 100]$; the `FILL_NULL` on line 4 needs marks without missing values; the `FILTER` on line 5 needs marks of at least 10 without missing values; the check on line 3 needs names. The *admission certificate*, which keeps only what no run-time code can stand in for, requires only the column types. The generated program keeps the four statements behind guards, computes missing-value counts and ranges of the columns the guards read, and for this batch re-enables only the check on line 2:

```
Deoptimized line 2: EXPECT s.marks BETWEEN 0 AND 100:
  marks goes up to 101, above 100
ContractError: line 2: s.marks has values outside 0 to 100
```

The run stops exactly where a fresh compilation of the batch stops (it reports the contract as violated). A batch with missing marks re-enables the fill and the filter, and writes what a fresh compilation writes; a batch with marks as text is refused, because the compiled code reads marks as integers. On 400 random batches, 127 fit the full certificate and 361 are admitted with deoptimization; for every one of them, a fresh compilation proves every removal the program keeps (Section 10).

Composition. The compiled program also states what `out.csv` satisfies for every admitted batch: marks in $[10, 100]$ without missing values (the deferred and re-enabled checks hold whenever the run finishes). A second stage compiled on `out.csv` needs no admission check for any fact that this guarantee implies, and `tools/contracts.py` names the facts it does not imply, with the `EXPECT` that stage 1 would need.

3 Data-Aware Compilation

3.1 Language

PrepLang programs are straight-line sequences of statements over tables (one CSV or Parquet file), table sets (a folder of them), images, image sets, and images linked to a table of labels. Table statements are `FILL_NULL` (with a value, `MEAN`, `MEDIAN` or `MODE`), `DROP_NULL`, `NORMALIZE`, `FILTER` (with `AND`, `OR`, `NOT` and comparisons), `SELECT`, `DROP`, `PARSE_DATE`, which turns a text column of ISO dates into DateTime, and `DERIVE t.c = e`, which adds or replaces a column computed with `+ - * /` from numeric columns and numbers, or as the `YEAR`, `MONTH`, `DAY`, `HOUR` or `WEEKDAY` of a date; image statements are `RESIZE`, `GRAYSCALE`, `NORMALIZE RANGE`, `CONVERT`, `COMPRESS` and `REMOVE_CORRUPT`. Selectors such as `company.*.* WHERE TYPE NUMERIC` address many columns, resolved against the inferred schemas. Contracts are written with `EXPECT`: `NUMBER`, `STRING`, `BOOLEAN`, `NOT_NULL`, `BETWEEN`, `UNIQUE`, `REFERENCES`, `SCHEMA_SAME_AS`, `CHANNELS`, `MIN_WIDTH`, `MIN_HEIGHT` and `MATCH_RATE`. Every statement begins with its own keyword, so the grammar is LL(1); the front end is a hand-written lexer and a recursive-descent parser with panic-mode recovery. The compiler generates one Python program (pandas for tables, OpenCV for images, one streaming pass per image save).

3.2 The abstract dataset state

The semantic analyzer maintains a state σ mapping every dataset to an abstract value; a single forward pass in the style of abstract interpretation [21] suffices, since programs have no loops. For a table T and column c, let $V_c(T)$ be the multiset of present values and $M_c(T)$ the number of missing ones. A column is abstracted as $a = \langle \tau, \nu, [l, u], \epsilon \rangle$: a type, $\nu \in \{none, maybe\}$ for missing values, an interval [22], and an exactness flag, with

$$\gamma(a) = \{\, T \mid V_c(T) \subseteq [l,u] \text{ with type } \tau,\ \nu = none \Rightarrow M_c(T) = 0,\ \epsilon \Rightarrow \big(M_c(T) > 0 \Leftrightarrow \nu = maybe\big) \wedge \big(V_c(T) = \emptyset \vee \min V_c = l \wedge \max V_c = u\big) \,\}.$$

An exact column promises that both bounds occur in the data, which is what allows a compile-time Violated verdict. `LOAD` of a CSV file builds the exact state from every cell (a parallel scanner analyses one million rows in 0.13 s, 4.7× faster than pandas reads them). An image set is abstracted as per-file shapes (width, height, channels, broken or not) from the PNG or JPEG header, a pixel type, a pixel range and an output format. Transfer functions follow the statements: `FILTER` refines intervals with its comparisons and, as in SQL's three-valued logic [23], makes compared columns non-missing; `FILL_NULL` clears *maybe* and joins the fill value; `NORMALIZE` sets $[0, 1]$; `DERIVE` computes the interval of its value by interval arithmetic and makes it *maybe* missing where an operand is; `RESIZE` sets every shape. `PARSE_DATE` reads the values of its column (only then, so no other program pays for it): if every value is a date in one ISO layout, the column becomes DateTime with the first and last date as its interval, and otherwise the compiler names the first row that is not a date. A part of a date has its full range (`MONTH` in $[1, 12]$) unless the dates fall in one year, month or day: dates from October to December give `MONTH` in $[10, 12]$, so a later `FILTER` on the month can be proven to keep every row. A division by a value whose interval contains 0 is a compile error, since pandas would write infinities or missing values without a word. Saves check that the format can hold the pixels: Float32 pixels cannot be saved as PNG, which a plain script would write as black images.

Metadata only. For Parquet files [25, 26] the compiler reads only the footer: types from the schema, intervals from the per-row-group statistics that databases use as zone maps [24], missing values from the null counts. Compilation then costs 0.018 s for one or 16 million rows; soundness rests on truthful writer statistics.

3.3 Hybrid contract checking

For a contract φ at a point with state σ, the compiler returns Proven if every dataset in $\gamma(\sigma)$ satisfies φ (no check is generated), Violated if none does (compilation fails, naming the first offending row), and Deferred otherwise (a run-time check is generated). This is hybrid type checking [19] and soft typing [20] applied to data. A deferred contract refines σ afterwards, because execution continues only if its check passed.

Lemma 3.1 (Soundness of the analysis). *If T is the content of a CSV input, or of a Parquet input with truthful statistics, then $T \in \gamma(\sigma_0)$; for every statement s, $T \in \gamma(\sigma) \Rightarrow [\![s]\!](T) \in \gamma(f_s(\sigma))$; a Proven verdict holds for every $T \in \gamma(\sigma)$ and a Violated one for none.*

Proof sketch. The initial state is computed from all cells, or from statistics that bound the values; exactness is claimed only when both bounds and the missing count are known. Transfer functions are case by case: `FILTER` keeps a subset of rows that satisfy the condition and clears ϵ; `FILL_NULL` adds its value exactly when values are missing. A Violated verdict needs an exact column, whose bound is a value of the data. □

Optimizations. Analysis results are attached to a three-address IR. *Check elimination* removes checks of proven contracts; *no-effect elimination* removes a `FILL_NULL` on a column without missing values, a `FILTER` that is always true, a `GRAYSCALE` of 1-channel images, a `RESIZE` to the size every image has; *column pruning* loads only live columns. All of them keep the output exact; the one approximate rewrite (merging two resizes) runs only on request. Contracts

proven by reading values (`UNIQUE`, `REFERENCES`, `MATCH_RATE`) keep a check that runs only when an input differs from the compiled one.

4 Data Certificates

Let σ_0 be the input state of the compiled files with every exactness flag cleared, so that it describes a set of files rather than one. For an input state ι, let $\Pi(\iota)$ be the set of *obligations* the analysis discharges from it: proven contracts and no-effect facts (each identified by its statement and target), leaving out proofs that read the values, plus the absence of compile errors. Order input states pointwise: *none* $\sqsubseteq$ *maybe*, interval inclusion, a fixed type, and for images per-file shapes inside boxes.

Lemma 4.1 (Monotonicity). *If $\iota' \sqsubseteq \iota$, then $\Pi(\iota') \supseteq \Pi(\iota)$.*

Proof sketch. Every decision is the inclusion of the state in a fixed predicate ($[l,u] \subseteq [a,b]$, $v = \mathit{none}$, a comparison true on all of $[l,u]$), which a smaller state keeps. Transfer functions are joins and meets with constants; where a decision changes the path (a filter always true in ι' but not in ι, a contract proven in ι' but deferred in ι), the state of ι' is below the refined state of ι because it already satisfies the condition. With exactness cleared, no verdict depends on a bound occurring in the data. □

A *certificate* for a set of obligations O is an input state C_O with $O \subseteq \Pi(C_O)$. An input T *fits* it if $\alpha(T) \sqsubseteq C_O$, where $\alpha(T)$ is T's own inexact state: every certified column has the type the compiler would infer, no missing values where the certificate says *none*, values inside the certified interval, and every image file inside its box.

Theorem 4.2 (Certificate soundness). *If T fits C_O, every obligation in O holds for T.*

Proof. $\Pi(\alpha(T)) \supseteq \Pi(C_O) \supseteq O$ by Lemma 4.1; by Lemma 3.1, each proof derived from $\alpha(T)$ holds for T at its program point, and each removed operation does nothing there. □

Counterfactual re-analysis. The certifier starts from σ_0 (first checking that it still discharges what the compiled program used; otherwise only the exact files are admitted) and weakens one fact at a time, running the whole analysis again: missing values are allowed; a lower bound becomes $-\infty$, or else the loosest program constant below it that works, first as a closed bound ($\geq k$) and then as an open one ($> k$); likewise upper bounds; a column the program only carries to its output may be absent; image boxes gain channel counts and size ranges, again up to the program's constants; the first and last date of a column `PARSE_DATE` reads become open, or else the start or end of their year, month or day; and that the column holds dates is kept or dropped like a type. A weakening is kept if the criterion still holds. Without `DERIVE`, the analysis compares intervals only with the program's constants, so no other bound can keep more and the result is 1-minimal (no remaining fact can be weakened on its own). A derived column compares values computed from its inputs (`temp - 273.15 > 0`), and a part of a date changes at calendar boundaries; there the result is maximal among the candidates, which include the numbers in `DERIVE`, but a bound between them could keep more. Either way a rejected weakening tells which obligations need the fact. The search uses the analysis as a black box, so it needs only a monotone forward analyzer, not backward operators [48]. When every obligation is a conjunction of conditions on single facts of one column (range and not-null contracts, fills, and filters over one column are), the certificates that discharge it form a product of chains, one per fact, and the search finds its greatest element whatever the order in which facts are tried; an obligation over several columns, such as a filter `a > 0 OR b > 0` that is always true, can have incomparable maximal certificates (`a > 0`, or `b > 0`), and the order picks one. Each run reads no data: tables are replaced by their facts and image folders by *corner images*, one per combination of the extreme sizes, channel counts and depths of the box. Every image decision is a universal or existential test over files of a conjunction of one-dimensional conditions, so a folder inside the box passes whenever its corners do.

Every column. The weakening runs over every column of the file, not only those the optimized program loads: once `EXPECT t.id NOT_NULL` is proven and pruning drops `id`, the certificate still requires it, and the generated program reads `id` only to check it.

5 Selective Deoptimization

The full certificate $\mathcal{F} = C_{\Pi(\sigma_0)}$ treats every removal alike, so a batch that breaks the fact behind one removed check is refused, although running that check would decide the batch correctly. Dynamic compilers face the same choice with speculative code and resolve it by deoptimization [34]. PrepLang applies it per batch and per removal.

Speculative removal. The optimizer keeps every removed check and operation in the IR, marked speculative (with the comparisons a removed `FILTER` needs). The certifier then finds, one after the other and each starting from the previous result:

(1) the *full certificate* $\mathcal{F}$, for all of $\Pi(\sigma_0)$;
(2) the *admission certificate* $\mathcal{A}$, for the obligations no run-time code can stand in for: the absence of compile errors, contracts without a run-time check (types, image contracts), and removals that cannot be re-enabled (a check of a column pruning dropped);
(3) an *item certificate* C_i for every speculative removal i, for the obligation $\{i\}$ and the absence of errors, starting from $\mathcal{F}$.

An item certificate is stored as the facts it needs beyond $\mathcal{A}$; one that needs none is removed for good. The generated program refuses a batch T that does not fit $\mathcal{A}$ (naming the fact and the obligation), computes the missing-value count, minimum and maximum of the columns item certificates mention (or the header facts of image folders), and runs each speculative instruction i only if T does not fit C_i:

```
if deopt("2:1:s..marks"):
    if not df_s["marks"].dropna().between(0, 100).all():
        raise ContractError(
            "line 2: s.marks has values outside 0 to 100")
```

For the compiled file, `deopt` is false without looking at the data.

Theorem 5.1 (Exact selective deoptimization). *Let T fit $\mathcal{A}$ and let $D = \{\, i \mid T \text{ does not fit } C_i \,\}$. The generated program with the*

removals outside D skipped and those in D run computes on T exactly what the source program computes: the same outputs, or a stop at the same check.

Proof. For $i \notin D$, $\alpha(T) \sqsubseteq C_i$ and $i \in \Pi(C_i)$, so by Lemma 4.1 $i \in \Pi(\alpha(T))$, and since T fits $\mathcal{A}$ no compile error occurs; by Lemma 3.1 instruction i does nothing at the state the source program reaches there on T. Removing instructions that do nothing along a run does not change the run: by induction on the program, the state before every remaining instruction is the same. The instructions in D and all others run as in the source program. □

Corollary 5.2 (Agreement with a fresh compilation). *If a fresh compilation of T succeeds, its program computes the same as the source program on T (Lemma 3.1), hence as the deoptimized program. If it reports a contract as violated, the removal of that contract's check is not in $\Pi(\alpha(T))$, so it is in D or was never removed; the source program fails on T at that check or at an earlier one, and so does the deoptimized program.*

So a batch is refused only for facts that no run-time code stands in for. Item certificates are weaker than $\mathcal{F}$ in the directions their removal does not care about: the check on line 2 of Fig. 2 needs no lower bound of 10 and allows missing marks.

Mechanization. We mechanized the theorem in Lean 4 [60] in two layers (`proofs/`, 1142 lines of Lean's core language, no Mathlib; the theorems use only the axioms `propext` and `Quot.sound`, which gives function extensionality, and no classical choice). `Deopt.lean` proves Theorem 5.1 for an abstract setting: a program is a list of partial state transformers; an analysis is any function from abstract inputs to discharged obligations that is monotone and whose no-effect proofs are sound for admitted inputs; item certificates are abstract inputs on which the analysis discharges their removal; and the run-time check is sound for $\sqsubseteq$. It also proves that the greedy search ends only on certificates the re-analysis accepted and that it only weakens. `Core.lean` instantiates the setting with the analysis of a core of PrepLang: numeric columns abstracted by v and an interval, `FILL_NULL`, `FILTER` with $\geq$ and $\leq$, `DROP_NULL`, `DERIVE` of the sum of two columns (interval addition), and `BETWEEN` and `NOT_NULL` contracts. For this core it proves the transfer functions sound and every Proven verdict and no-effect proof valid (Lemma 3.1), monotonicity (Lemma 4.1), the run-time check of the generated code (missing values and bounds of the certified columns) sound and complete for a certificate, and from these Theorem 5.1 and the composition theorem of Section 6 end to end. For the rest of the language (types, text, normalization, other arithmetic, dates, images, contracts proven by reading values) the two lemmas remain assumptions, which we test with randomized property tests that compare deoptimization decisions with fresh compilations (Section 10.9).

6 Composing Pipeline Stages

Pipelines have stages: a cleaning stage writes a table that a feature stage reads. Each compiled stage records, for every table it saves, the state the analysis computes at the `SAVE` when its inputs are given the admission certificate: the stage's *guarantee* G. Because selective deoptimization keeps the source program's semantics (Theorem 5.1) and the analysis is monotone, every file the stage writes for an admitted batch has its facts inside G; facts of deferred contracts and of re-enabled checks hold because the run passed them.

Theorem 6.1 (Composition). *Let stage 2 read the table written by stage 1. If $G \sqsubseteq C_i$ for a removal i of stage 2, stage 2 never deoptimizes i for an output of stage 1 on an admitted batch; if $G \sqsubseteq \mathcal{A}_2$ on value facts, stage 2 never refuses such an output for its values.*

The theorem is transitivity of $\sqsubseteq$ once the facts survive the file between the stages; values and missing values do, while a type can change in a CSV round trip (a column whose values are all missing reads back as text), so composition covers value facts and types are still checked on admission, from metadata. `tools/contracts.py` reads the contract file every compilation writes (`preplang_contract.json`) and reports, for each fact stage 2 relies on, whether G implies it; for each fact it does not, it writes the `EXPECT` statement that stage 1 would need before its `SAVE`. Given several consumers of one table, it computes the conjunction of their requirements: a *producer contract*, synthesized from the consumers' code, under which no consumer ever refuses or deoptimizes. Assume-guarantee reasoning is standard for component contracts [56], and schema registries check the containment of writer and reader schemas [57]; here the contracts on both sides are derived from the stages' proofs.

7 Compiling Against Data You Cannot See

A data-aware compiler leaks its data through what it writes. The generated program embeds the checksum of the compiled file, which confirms a guessed file; every removal states a fact about it (no value of `age` is missing, no mark is below 10); a violated contract names a row; the contract file records bounds. Yet programs travel further than data: in validation servers [68] and platforms such as OpenSAFELY [74], an analyst writes code for confidential records against dummy data, and the data owner runs it on the real ones. The analyst needs what a data-aware compiler gives (which fills are needed, which contracts the data breaks, which filters do nothing) and may not see the data that produces it.

Guards are necessary. The facts such a compiler needs are *emptiness* facts: no value is missing, no value is below k. No differentially private release can certify one. If a mechanism M is ε-differentially private and a table D' differs from D in k rows that break the fact, then $\Pr[M(D') = o] \geq e^{-k\varepsilon} \Pr[M(D) = o]$ for every output o: whatever M says on a clean table it also says, with comparable probability, on a table that breaks the fact in a few rows. One-sided noise keeps differentially private query processing exact when it decides how many padding tuples to drop [73, 75], up to the failure probability δ of approximate differential privacy; it cannot make an *unguarded* removal safe, because a removal needs the emptiness fact itself. A program optimized from released emptiness facts must check them on the data it runs on. Selective deoptimization does exactly that. Theorem 5.1 has no hypothesis about the facts the compiler starts from (in Lean, `deopt_correct` quantifies over arbitrary removals and certificates): the facts decide which instructions are removed speculatively and which certificates guard them, and the generated program checks the certificates on the batch. A wrong fact costs a deoptimization, never a wrong result.

What is released. The data owner runs the compiler with `-queries`, which writes the schema of each input (names and types of its columns) and the numbers written in the program. The analysis compares bounds only with these numbers (Section 4): for a program without `DERIVE`, the numbers $k_1 < \cdots < k_m$ (and 0) cut a column into the cells $(-\infty, k_1)$, $\{k_1\}$, (k_1, k_2), …, $\{k_m\}$, (k_m, ∞) and *missing*, and every decision depends only on which cells hold some row. `tools/release_facts.py` releases one Laplace histogram over these cells per column [64]: a row lies in exactly one cell of a column, so the histogram has sensitivity 1. Only columns the program names are asked about (all if it uses `*` or a `DROP_NULL` of whole rows), and of those only missing values unless they are numeric. Each table gets the whole budget ε, divided equally between its columns asked about. A cell counts as occupied if its noisy count is above $t + \frac{1}{2}$ with $t = \ln(C/\beta)/\varepsilon_c$, where C is the number of cells released and ε_c the budget of the cell's column. A column's facts are read off its occupied cells: missing values if the missing cell is occupied; bounds from the lowest and highest occupied value cells; and, as evidence, that values below the upper edge of the lowest occupied cell (or above the lower edge of the highest) exist.

Theorem 7.1 (Privacy). *For a schema fixed independently of the data, the facts file is ε-differentially private for adding or removing one row of any input table, and so is everything the compiler writes for the analyst: the generated program, the certificates, the contract file, the manifest and every message.*

Proof. A Laplace histogram with noise $\mathrm{Lap}(1/\varepsilon_c)$ per cell and sensitivity 1 is ε_c-differentially private. A row is in every column of its table, so those columns compose sequentially, with $\sum \varepsilon_c = \varepsilon$ per table; a neighbouring dataset differs in one row of one table, so the tables compose in parallel. Which columns and cells are asked about depends only on the program and the schema, never on an answer, so the composition is non-adaptive. The occupied cells, the facts, the evidence and the compiler's outputs are computed from the noisy histograms, the program and the schema alone; the compiler never opens an input file (`-facts` fails rather than read a table without released facts). □

The hypothesis on the schema is not a formality. A type the compiler *infers* is a fact about the values: a CSV column is Integer until one cell holds 3.5, and pandas writes an integer column with a missing value to Parquet as a float column. `-queries` writes the inferred types as the owner's draft; the owner must publish the schema as declared (a data dictionary), not as found. The seed of the noise is never written to the facts file: whoever knows it can recompute the noise.

Theorem 7.2 (Exactness for any facts). *For any facts file, true, noisy or false, the generated program computes on every batch it admits exactly what the source program computes.*

Proof. Without a checksum, no batch is treated as the compiled one: every batch, including the file the facts describe, is checked against the admission and item certificates. Theorem 5.1 applies unchanged. □

What the analyst sees of a run. On a validation server the analyst learns at least whether a job finished. If a released fact could make the generated program refuse a batch, an analyst who chooses the program's numbers could learn a bit about one row from each submission. So no released fact enters the admission certificate: in a private compilation a removed check keeps its column loaded (a normal compilation prunes the column and makes the check's proof an admission fact), so every removal can run again. The admission facts that remain are the declared types and the absence of compile errors.

Corollary 7.3 (Outcomes do not depend on the facts). *On every batch whose fresh compilation succeeds, the generated program finishes, with the source program's output, exactly when the source program finishes. Of a run, the released facts change only which removed instructions run again, that is, how long it takes.*

Proof. Types are declared, not released. A batch that breaks the facts behind the absence of compile errors also makes its fresh compilation fail, or else the admission certificate was stricter than needed, which can only refuse a batch the source program would run (we never observed it). Every other fact guards a removal that runs again when broken (Theorem 7.2). □

Lemma 7.4 (What the analyst learns). *With probability at least $1 - \beta$ over the noise, for every released cell: (i) if it is occupied, it holds at least one row; and (ii) if it holds at least $2t + 1$ rows, it is occupied. Hence every "likely violated" warning names a contract the data breaks; for a program without `DERIVE` whose columns each have an occupied value cell, every released bound is at or inside the tightest true bound at the program's numbers (an occupied cell holds a row), so the private compilation removes at least what a normal compilation of the same file removes (except checks of `UNIQUE` and `REFERENCES`, which read values); and a released fact is false only if fewer than $c(2t + 1)$ rows break it, where c is the number of cells it covers ($c = 1$ for "no missing values").*

Proof. $\Pr[|\mathrm{Lap}(1/\varepsilon_c)| > t] = e^{-\varepsilon_c t} = \beta/C$, so by a union bound every noise is at most t with probability $1 - \beta$. Then a noisy count above $t + \frac{1}{2}$ has a true count above $\frac{1}{2}$, and a true count of at least $2t+1$ gives a noisy count of at least $t+1$. A column with no occupied value cell gets no bounds at all, which is why the hypothesis is needed. The removals follow from Lemma 4.1, because decisions compare bounds only with the program's numbers. □

The bound in the last part is what separates the release from the *optimistic* facts "nothing is missing", which need no data and also give every removal of a normal compilation. False facts cost deoptimizations, and Theorem 7.2 keeps every admitted run exact. In a normal compilation a false admission fact (the proven check of a column the optimized program does not load) would instead refuse batches; a private compilation keeps such checks (Corollary 7.3). The mechanized core does not cover admission (its admission predicate is true), types or compile errors; those parts are tested. Because the release depends only on which cells are occupied, one release can serve several programs over the same files: the cells of their numbers together are released once, and each program reads its facts off them. A private compilation cannot report a contract as violated, since nothing it knows is exact and a row would name

a person; a contract the released evidence shows broken gets a warning ("`EXPECT s.marks BETWEEN 0 AND 100` is likely violated: the released facts report values of `s.marks` above 100"), and the check runs on the real data. Contracts that read the values (`UNIQUE`, `REFERENCES`) are always checked at run time. A fact about a few rows, such as two missing `Embarked` values among 891 passengers, is hidden by design; the run-time check covers it. Images, folders of tables and `PARSE_DATE`, which read values or file lists, are not supported in a private compilation. The guarantee covers what the compiler writes: what the owner's runs print (a deoptimization names a row) and how long they take stay with the owner, and releasing them would need a mechanism of its own.

8 Lifting pandas Scripts

Certificates are most useful for code that already exists. A second front end parses a pandas script with Python's own parser, follows every DataFrame read with `pd.read_csv(path)`, and translates, in order, the statements that change a followed DataFrame in a way PrepLang expresses: `fillna` with a constant or the column's `mean()`, `median()` or `mode()[0]`; `dropna`; `drop` of columns; column selection; boolean-mask filters; min–max and z-score normalization written as arithmetic; `assert` with `between`, `notna` or `is_unique`; and `to_csv`. A second version, written after the corpora of Section 10.5 were evaluated, also translates `pd.to_datetime` of a column into `PARSE_DATE` (with `errors="coerce"`, too: on the inputs PrepLang admits, both read the same dates), `.dt.hour` and the other date parts, and arithmetic over columns and numbers into `DERIVE`. Statements that only read a DataFrame are skipped. Lifting stops at the first statement that changes a DataFrame in another way (`map`, `get_dummies`, `set_index`): the result is the longest prefix of the script's cleaning that PrepLang expresses exactly. The lifted statements, run on their own, are the ground truth for the compiled program in Section 10.

9 Implementation

The compiler is about 9,112 lines of C++17 without external libraries: lexer, parser, schema provider (parallel CSV scanner, a Thrift compact-protocol reader for Parquet footers, PNG and JPEG header readers), semantic analyzer, IR, pass manager, certifier, and an execution planner over the pandas and OpenCV code generators. Every compilation writes the generated program, a manifest, and the contract file. The front end for pandas, the composition tool and the evaluation harness are Python; the unit tests (281 checks in 9 suites) include the randomized soundness tests.

10 Evaluation

We ask seven questions. **RQ1:** Does compiling against the data find mistakes before any data is processed? **RQ2:** What does data-aware compilation cost and gain? **RQ3:** Can a compiled program be reused on edited batches? **RQ4:** On real data that changes from month to month, compared with data-validation tools? **RQ5:** For real pandas scripts? **RQ6:** Do certificates compose across stages? **RQ7:** What do certification and reuse cost? We also report how we tested soundness and the bugs the evaluation found in our own implementation.

Setup. Intel Core i5-12450H (8 cores), 16 GB RAM, Windows 11; MSVC 14.44 (Release); Python 3.14, pandas 3.0, NumPy 2.5, OpenCV 5.0, pyarrow 25. pandera 0.34 and Great Expectations 0.18.22, whose onboarding data assistant builds a suite from a sample (we found no such assistant in the 1.x line), run in a Python 3.11 environment, TFDV 1.21 in WSL (Ubuntu, Python 3.12), since it has no Windows build. Times include starting Python.

Scoring reuse. RQ3–RQ6 score every way of running a new batch T against the ground truth: a fresh compilation of the pipeline for T, run. A policy either runs a program on T or refuses it. Its outcome is *correct* if it wrote exactly the files of the fresh run or stopped where the fresh run stops (a refusal, or a failing check, when the fresh compilation fails or its run stops); *needless* if it refused T although the fresh compilation runs, which costs a recompilation; and *wrong* if it finished with other files than the fresh run, finished where the fresh run fails, or stopped where it does not. The policies are: the *fingerprint* of the compiled file; the compiled file's own facts (types, no missing values, and the range of every column read, a schema as TFDV would record it with ranges); pandera's `infer_schema` (dtypes, nullability, numeric ranges); Great Expectations' onboarding data assistant (an inferred suite of column statistics); TFDV's `infer_schema` (types, presence, string domains); the *full certificate*; and *selective deoptimization*. The validators are set up on the file the pipeline was compiled with, once on every column (as they are used) and once on the columns the program reads (more favourable to them); when a validator admits T, the optimized program runs on it as it is.

10.1 RQ1: Mistakes found before any data is processed

Injected mistakes. We injected 24 mistakes into the five example programs, one per mutant [29]: misspelled columns, type errors (normalizing text, comparing a number with text), uses after removal, broken contracts (range, not-null, foreign key, schema, channels, width, match rate), a broken image, and three well-typed programs with silently wrong output. PrepLang reports all 24 at compile time and nothing on the unmodified programs, in 22–39 ms; a script-style interpreter of the same programs crashes part-way on 20 and finishes 4 with wrong output (text filled into an integer column, a threshold on a normalized column that removes every row, and two pipelines that write black images). On 16 million Parquet rows, a misspelled column is reported in 0.02 s instead of after 2.81 s, and float pixels saved as PNG in 0.26 s for 3,000 images, where the script runs 8.54 s and writes black images.

Real failures. We chose 17 error messages that Python libraries print for common data-preparation failures, searched GitHub issues for each and read the first 12 results in order; an issue qualified if the failure happened while loading, transforming or saving data files and its root cause was stated, at most three per message: 44 issues [30, 31]. PrepLang reports 18 of them (41%) before any data is processed (12 without annotation, 6 once the requirement is declared with `EXPECT`); 3 cannot occur in a PrepLang program, and 23 are not covered (input formats such as HEIC or GeoTIFF, operations such as pivots, dates, per-image label files). 21 of 21 reproductions behave as classified.

10.2 RQ2: Cost and gain of data-aware compilation

Analysing every cell of a one-million-row CSV file takes 0.13 s, 4.7× faster than pandas' default reader reads it; Parquet compilation stays at 0.018 s from one to 16 million rows; reading 3,000 image headers is 17× faster than decoding the images. On a 200,000-row, 40-column table with two contracts and a filter, the optimizer proves both contracts and loads 5 of 40 columns; the optimized program processes the data in 0.47 s instead of 0.79 s (40% less) with byte-identical output. Image pipelines stream one image at a time, in 10 MiB of memory instead of 922 MiB for loading all 1,000 images.

10.3 RQ3: Reusing a compiled program on edited batches

Seven workloads are compiled once: the four table examples, one program run twice on one file, the Parquet example, images linked to labels, and a sensor pipeline of 20,000 rows per day. Each input table gets new batches: other days and a warmer season of the sensor process, and, for the examples, rows resampled with replacement, values moved by noise of 0.1, 0.5 and 1.5 standard deviations, 20% missing values in one column, and one value far out of range (244 batches).

Both certificates handle every batch correctly (Table 1): the full certificate by refusing exactly the batches whose fresh compilation fails, selective deoptimization by running them and stopping at the re-enabled check (35 batches ran with removals re-enabled). The fingerprint refuses every edited batch (129 of them needlessly), and the compiled file's own facts refuse 81 that a fresh compilation runs. Gating the optimized program with the file's own facts is also unsound (41 wrong runs): on the table set and on the linked images, resampled and drifted rows repeat keys or lose matches, so a contract the compiler had proven by reading the values (`UNIQUE`, `REFERENCES`, `MATCH_RATE`) no longer holds and the fresh compilation reports it, while the file's facts, which are facts of single columns, admit the batch and the optimized program finishes. PrepLang runs such checks again whenever an input changed. The real validators fare no better: pandera's inferred schema is unsound on the same batches (41 wrong), TFDV's, which has no ranges, on these and on batches whose values drift past what a removed check or filter relied on (66 wrong), and Great Expectations' statistical suite, which is sound here, refuses 106 batches that a fresh compilation runs.

Image folders. A pipeline that makes images gray and 64×64, compiled on a folder of gray 64×64 images, has its `GRAYSCALE` and `RESIZE` removed speculatively. We gave it 30 new folders: gray 64×64 images, some color images, other sizes, both, a file that is not an image, and 16-bit images (five each), and compared the written images byte by byte with a fresh compilation. Selective deoptimization handles 25 correctly, re-running `GRAYSCALE` or `RESIZE` on 15 folders; the full certificate refuses those folders, and the optimized program run on them without a check writes color or wrongly sized images (15 folders). The 5 needless refusals are the 16-bit folders: bit depths are fixed in every certificate because they decide the pixel type the generated code assumes, even where, as here, no instruction depends on it; certifying them would need the code generator's assumptions as obligations too.

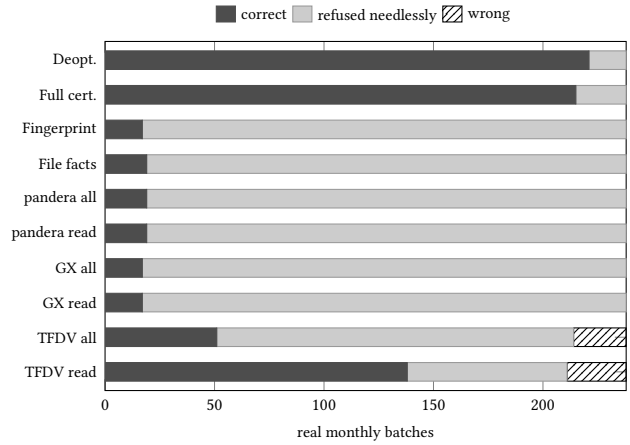


Figure 3: RQ4: 238 real monthly batches, by admission policy. *Correct*: ran and wrote the fresh compilation's files, or stopped where it stops; *refused needlessly*: a recompilation was needed although the fresh compilation runs; *wrong*: other files, or a run where the fresh compilation fails. Validators gate the optimized program; *all*/*read*: set up on every column, or on the columns the program reads.

10.4 RQ4: Real monthly data, against data-validation tools

Three public datasets come in monthly batches: hourly traffic and weather on a Minnesota interstate (UCI, 2012–2018, 63 months of CSV), daily bike rentals in Washington (UCI, 2011–2012, 24 months), and New York green-taxi trips (NYC TLC, 2022–2024, the 36 Parquet files as published). For each we wrote one cleaning pipeline as a data engineer would: range contracts on the measured quantities, `NOT_NULL` on keys, fills, a filter that removes impossible readings, a normalization and a selection. Each pipeline is deployed twice: compiled on the first month and run on every later one, and compiled on the month before each month (238 runs; twice the month before has a temperature of 0 K, fails to compile itself, and leaves nothing to deploy). Batches are cut from the raw files, so no value is reformatted.

Over 238 runs (Table 2, Figs. 3 and 4), selective deoptimization handles 221 (93%) exactly as a fresh compilation would, with 0 wrong; the full certificate handles 215. The difference is 9 months the full certificate refuses. In three, the traffic sensor reports temperatures of 0 K; the re-enabled range check stops the run, as the fresh compilation fails. In four, a month has an hour with 9,831 mm of rain or a day with humidity 0, which the compiled month had not; the removed cleaning filter runs again and the run writes what the fresh compilation writes. In two, a taxi trip has a fare of $2,020; the re-enabled check runs after the filter that removes that trip, passes, and the run finishes. All 17 remaining refusals are one event: in February 2023 the taxi publisher changed `passenger_count` and `payment_type` from double to integer, and the program compiled

Table 1: Reusing compiled programs on edited batches (RQ3): per policy, batches handled correctly (ok: the fresh files, or a stop where the fresh run stops), refused although a fresh compilation runs (ref.), and finished with other files or where the fresh run fails (wrong).

	tables			images + labels			all		
Policy	ok	ref.	wrong	ok	ref.	wrong	ok	ref.	wrong
Fingerprint	98	126	0	17	3	0	115	129	0
File facts	121	78	25	1	3	16	122	81	41
pandera, every column	120	79	25	1	3	16	121	82	41
pandera, columns read	123	76	25	1	3	16	124	79	41
Great Expectations, every column	121	103	0	17	3	0	138	106	0
Great Expectations, columns read	123	101	0	17	3	0	140	104	0
TFDV, every column	146	28	50	1	3	16	147	31	66
TFDV, columns read	149	25	50	1	3	16	150	28	66
Full certificate	224	0	0	20	0	0	244	0	0
Selective deoptimization	**224**	**0**	**0**	**20**	**0**	**0**	**244**	**0**	**0**

Table 2: Reusing compiled pipelines on real monthly batches (RQ4), both deployments (compiled on the first month, and on the month before). ok: handled as a fresh compilation of the month would; ref.: refused although a fresh compilation runs; wrong: finished with other files than the fresh run, or where it fails. Validators gate the optimized program (Section 10.4).

	Traffic (CSV)			Bikes (CSV)			Taxi (Parquet)			all 238		
Policy	ok	ref.	wrong	ok	ref.	wrong	ok	ref.	wrong	ok	ref.	wrong
Fingerprint	3	119	0	0	46	0	14	56	0	17	221	0
File facts	5	117	0	0	46	0	14	56	0	19	219	0
pandera, every column	5	117	0	0	46	0	14	56	0	19	219	0
pandera, columns read	5	117	0	0	46	0	14	56	0	19	219	0
Great Expectations, every column	3	119	0	0	46	0	14	56	0	17	221	0
Great Expectations, columns read	3	119	0	0	46	0	14	56	0	17	221	0
TFDV, every column	5	117	0	0	46	0	46	0	24	51	163	24
TFDV, columns read	92	27	3	0	46	0	46	0	24	138	73	27
Full certificate	120	2	0	44	2	0	51	19	0	215	23	0
Selective deoptimization	**122**	**0**	**0**	**46**	**0**	**0**	**53**	**17**	**0**	**221**	**17**	**0**

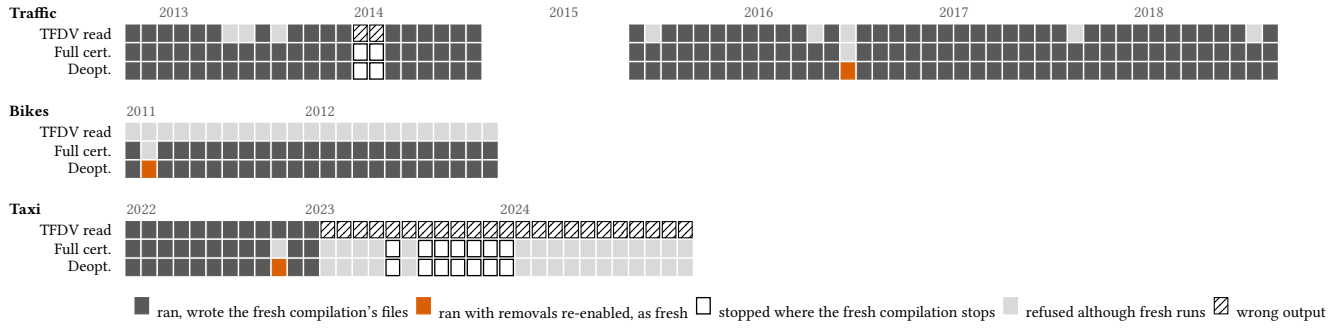


Figure 4: RQ4, compiled on the first month: every later month on a calendar axis (the traffic data has a gap), for the best validator, the full certificate and selective deoptimization. White: a fresh compilation stops too (temperatures of 0 K in 2014, taxi fares above $1,000). The grey run in the taxi rows begins in February 2023, when the publisher changed two column types; TFDV admits those months, and the optimized program writes them with the old types.

on January 2022 reads them as Float. A type is a fact no run-time code of the compiled program stands in for; monthly compilation follows the change after one month.

The validators are either strict or unsound. Great Expectations' profile constrains means, medians, quantiles and unique counts, and pandera's inferred schema the compiled month's range of every column; set up on every column or only on the columns read, they refuse 92–93% of the months although a fresh compilation runs them. TFDV infers types, presence and string domains but no numeric ranges. On every column it refuses 68% of the months, for string values of columns the program never reads (a new holiday name refuses a traffic month); on the columns read it refuses fewer, but both setups admit months on which the optimized program does not do what a fresh compilation does (27 months): taxi months after the type change, written with the old column types, and months on which a removed range check should have stopped the run, taxi

Table 3: Real pandas scripts from GitHub (RQ5): held-out scripts, lifted, compiled once on the training file and run on new batches; the ground truth is the lifted pandas statements run on the same batch.

	Titanic	House Prices
Held-out scripts	272	461
Lifted, in scope	130	168
Compiled	124	168
Runs on new batches	1364	1848
Full certificate: admitted	1347	1838
identical to the script	1347	1838
refused, script runs	10	3
With deoptimization: admitted	1357	1841
identical to the script	1357	1841
stopped by a check	0	0
different (unsound)	0	0
refused, script runs	0	0
ran with removals re-enabled	10	3

fares above $1,000 and traffic temperatures of 0 K, where the optimized program finishes and writes the bad rows. Generic validators do not know which facts the compiled program relies on.

10.5 RQ5: Real pandas scripts

Corpora. GitHub code search for Python files containing `read_csv`, `fillna` and `Embarked` (the Kaggle Titanic data) returned 372 distinct files, and for `read_csv`, `fillna` and `LotFrontage` (Kaggle House Prices) 461 files. We used the first 100 Titanic scripts as a development set, to choose which idioms the lifter handles; the remaining 272 and all House Prices scripts are held out: we did not look at them before the final run, and the lifter was not changed for House Prices. A script is in scope if it reads the training file with `pd.read_csv(path)` at module level. The House Prices files mark missing values with the text `NA`, and one column has a category called `None`; pandas reads both as missing. We give both sides the same files, with an empty cell, PrepLang's notation, wherever pandas reads a missing value. Our first run replaced only `NA`: two scripts then differed from pandas on every batch, the training file included, because PrepLang read `None` as a value that the scripts fill. That is a difference of CSV notation, independent of reuse; an exact lifter must carry pandas' list of missing-value strings into `LOAD`.

Protocol. The lifted program is compiled against the training file and given eleven new batches: Kaggle's real test file (no target column, and missing values in columns where the training file has none), five bootstrap resamples of the training rows, and five copies with 20% extra missing values in one column. Each batch is run by the compiled program and, as ground truth, by the lifted pandas statements; the cleaned tables are compared value by value.

Results. With deoptimization, 1357 Titanic runs and 1841 House Prices runs reproduce the scripts' results exactly, and none differ (Table 3). The difference to the full certificate is the price of optimizing for the compiled data: the training files have no missing `Fare` (Titanic) or `MSZoning` and `GarageCars` (House Prices), so the compiler removes the scripts' `fillna` on them, and the full certificate then refuses every batch with missing values there, including Kaggle's own test files. Deoptimization runs the removed `fillna` again for exactly those batches (10 Titanic and 3 House Prices runs). Most House Prices scripts fill columns that already have missing values in the training file, such as `LotFrontage`; those fills are never removed, and the certificate does not constrain them. The remaining refusals are batches the scripts crash on too: Kaggle's test files have no target column. The second lifter version (Section 8), written on other scripts after these corpora were evaluated, lifts 462 cleaning statements of the same held-out scripts instead of 397; the runs end as before (3198 of 3212 identical to the scripts, 0 different).

Table 4: Other people's pandas scripts for the monthly traffic and bike data (RQ5): lifted with derived columns and dates, compiled on the first month and run on every later month; the ground truth is the lifted pandas statements run on the same month. The traffic scripts were the development set of the second lifter version.

	Traffic (development)	Bikes, daily (held out)	Bikes, hourly (held out)
GitHub files	241	254	450
lifted, in scope	42	19	12
compiled on the first month	41	19	12
with a removal that can run again	3	0	1
statements lifted	104	38	21
of them `DERIVE`	51	22	7
of them `PARSE_DATE`	36	9	7
runs on later months	2,542	437	276
With deoptimization: as the script	2,542	437	276
stopped where the script fails	0	0	0
refused, script runs	0	0	0
different (wrong)	0	0	0
ran with removals re-enabled	6	0	0
Full certificate: as the script	2,536	437	276
refused, script runs	6	0	0
different (wrong)	0	0	0

Other people's pipelines on real monthly data. The pipelines of RQ4 are ours, and most Kaggle batches above are made, not observed. GitHub code search for Python files that read the UCI traffic file, or the bike-sharing `day.csv` or `hour.csv`, returned 945 files. Their cleaning mostly begins by parsing the date column and deriving hours, weekdays or Celsius temperatures from it, which the first lifter version cannot express. With `PARSE_DATE` and `DERIVE` in the language and the second lifter version, developed on the traffic scripts while the bike scripts stayed held out, 72 scripts compile on the first month of the real data (80 `DERIVE` and 52 `PARSE_DATE` among 163 lifted statements) and run on every later month (Table 4). All 3255 runs end as the scripts' own statements do on the same month, and none is refused. Most of these programs have no proof that depends on values that drift: their certificates require column types, and for 51 of them that a column holds dates; 4 have a removal that can run again. Two traffic scripts filter impossible readings themselves (`temp > 0`, `rain_1h < 500`). On October 2012 both filters keep every row and are removed, and they run again in exactly the three months with a temperature of 0 K or 9,831 mm of rain, where the full certificate refuses the month (6 runs). One lifted script fills a text column with the number 0, which pandas accepts and PrepLang rejects as a type error.

Table 5: Composing two compiled stages on real monthly data (RQ6). Facts: what stage 2 relies on (admission and removals); Guar.: implied by stage 1's guarantee; Months: run through both stages; Deopt.: months stage 2 deoptimized; by a guaranteed fact: never.

Data	Stage 1	Facts	Guar.	Months	Deopt.	Guar. broken
taxi	as written	13	11	15	0	0
taxi	strengthened	13	13	15	0	0
traffic	as written	14	12	60	0	0
traffic	strengthened	14	14	60	0	0
bikes	as written	11	10	23	0	0
bikes	strengthened	11	11	23	0	0

10.6 RQ6: Composing stages

Each workload of RQ4 gets a second stage that reads the cleaned table, as a feature step would: range and not-null contracts, a fill, a filter and a normalization. Both stages are compiled once (stage 2 on what stage 1 writes for the anchor month), and `tools/contracts.py` compares stage 1's guarantee with what stage 2 relies on. Every later month runs through both compiled stages, and stage 2 is also compiled and run fresh on stage 1's output.

The composition shows that stage 1 already guarantees most of what stage 2 relies on (33 of 38 facts across the three workloads), so stage 2's checks of those facts can never fire on stage 1's output (Table 5). The facts it reports as open are genuine gaps between the stages: stage 1 never bounds taxi tips, which stage 2 expects in $[0, 500]$; it fills missing rain with 0 but never bounds rain from below; and it never checks the traffic cloud cover or the bike temperatures for missing values, which stage 2's filters, removed as always true, rely on. For each, the tool wrote the `EXPECT` that stage 1 would need. Across 196 months through both stages (in both versions), no fact reported as guaranteed ever made stage 2 refuse or deoptimize. The open facts also held in these months, so stage 2 never deoptimized at all; the composition's value here is that after the suggested statements are added to stage 1, every fact stage 2 relies on is guaranteed statically, for all future months, and adding them made no month stop earlier in stage 1 (the same 9 months stop there in both versions, all at the checks stage 1 already had). Every stage-2 run ended exactly like a fresh compilation of stage 2 on the same input.

10.7 RQ7: The cost of certification and reuse

Reusing a compiled program costs its admission check and, with deoptimization, the facts of the columns item certificates mention. Over the 92 real months on which every variant finished (median of three runs each), the median run took 0.88 s with no check at all, 0.88 s with admission against the full certificate, and 0.88 s with admission and deoptimization. Admission with deoptimization changed the run time by −1% at the median (interquartile range −3% to +1%), which is inside the run-to-run noise of the machine: the check is one pass over a few columns the program reads anyway. Compiling a month afresh took 28 ms before running and 0.94 s with the run.

Certification is paid once, at compile time. Over 11 programs (the shipped examples and the three real pipelines), the compiler ran 14 to 207 re-analyses with deoptimization against at most 122 for the full certificate alone (at most 2.3× as many), and every compilation took 21 to 35 ms. The analysis is cheap because it runs on dataset states, not on data.

The certificates are small enough to read. For the 8 table programs, counting a type, the absence of missing values and each bound as one fact, the full certificate has 6 to 17 facts, the admission certificate here holds only the type of each certified column, and each of the 23 item certificates needs 1 or 2 facts beyond it. The real pipelines' certificates mention 6 of the 9 traffic columns, 7 of 16 bike columns and 9 of 20 taxi columns, with bounds only at the program's constants. The validators constrain every column they are set up on with facts no proof needs, such as the compiled month's exact range or mean; that, not the number of columns, is why Great Expectations and pandera refuse as often on the columns read as on all of them.

We state the consequence plainly: at these sizes reuse does not save time, because a fresh compilation is itself cheap. What reuse buys is a fixed, reviewed artifact that runs unchanged on every admitted batch (§10.4), an exact signal when a batch drifts, naming the removal it breaks, and output guarantees a downstream stage can rely on without seeing the data (§10.6). Recompiling gives none of these: each month gets a different program and no statement about what it assumed.

10.8 RQ8: Compiling against data you cannot see

Protocol. For each RQ4 pipeline (compiled for its first month) and each of the 184 RQ5 Titanic scripts that compile (compiled for the training file), the data owner ran `-queries` and released facts about the compiled file, and the program was compiled with `-facts` in a folder that holds only the program and the facts file. The facts came from the release at the program's cells with $\varepsilon \in \{0.1, 1, 10, 100\}$ (ε is the budget of each table a program loads, $\beta = 0.05$; five seeds for the pipelines, one for the scripts, so the script rows spend one release per script on the same training file); from one release for all Titanic scripts together (two seeds); from the same cells without noise; and from four baselines that do not use the program's numbers: *generic* bins (the same budget per column on a fixed logarithmic grid, $[0.001, 0.01), \ldots, [10^5, 10^6)$, the negative ones and the outer ones, read the same way), *synthetic* data (a table sampled from the generic release, values uniform in occupied bins, compiled normally, as an analyst would who develops against differentially private synthetic or dummy data), *optimistic* facts (no value is missing, chosen without the data), and no facts at all. Random facts, true or false at random, test exactness. Every generated program was run on every batch of RQ4 or RQ5 and on the compiled file itself, and scored against a fresh compilation of the batch.

Exactness. 2,919 compilations ran 38,409 times; 0 runs were unsound or stopped wrongly, for every kind of facts, including the 398 compilations from random facts (5,646 runs). All 688 refusals of batches a fresh compilation runs are taxi months with the schema change of February 2023, which the normal compilation refuses too (§10.4). The unit test of the private compilation checks the same property on 300 random facts files with random inputs, 128 of which deoptimize.

What the analyst learns. Table 6 compares the removals of each compilation with those of the normal compilation of the same file. Without noise, the program's cells give exactly the normal removals (recall 100%): the analysis compares bounds only with those numbers. Recall alone is cheap: the optimistic facts reach 60% of it without data, at a precision of 11%. With the program's cells at $\varepsilon = 1$, the taxi and traffic pipelines keep all normal removals at full precision, while the generic bins, at the same budget, keep 60% (taxi) and 83% (traffic): the program's numbers (`BETWEEN 1 AND 265`, `temp BETWEEN 230 AND 320`) do not fall on the grid. Synthetic data compiled normally is worse: none of the 15 compilations of the three pipelines from synthetic data succeeds, at $\varepsilon = 1$ or 10; each stops with contracts the real data satisfies (a temperature of 867 K, sampled uniformly in its occupied bin), so the analyst gets errors where the private compilation gives a program. What can be learned depends on how many rows a fact concerns: a median month has 805 traffic rows, 31 bike rows and 64,204 taxi trips, and the bike months, one row a day, keep the range removals only at $\varepsilon = 10$. On the Titanic training file, removal precision rises from 17% at $\varepsilon = 1$ to 31% at $\varepsilon = 10$ (generic bins and synthetic data: 20%); the false facts are mostly "no missing `Embarked`", whose two missing values no release can reveal, and the run on the training file deoptimizes them. One release for all 184 scripts at a total of $\varepsilon = 1$ gives each of them 17% precision, about what a script gets from its own release at $\varepsilon = 1$ (17%), which together spend 184 times the budget: the programs share the cells of the columns they all use.

Warnings. On a 1,000,000-row Parquet file we injected a mistake into m rows (amounts above the contract's bound, missing values in a `NOT_NULL` column, values below a bound) (Table 7). A normal compilation reports the broken contract for every $m > 0$. A private compilation from the program's cells warns that it is likely violated in every run from $m = 100$ at $\varepsilon = 1$ and $m = 10$ at $\varepsilon = 10$, with 0 warnings about a contract the data does not break in 1818 compilations, as Lemma 7.4 predicts. Generic bins, and synthetic data run on the program (as on a validation server, where a run stops at its first failing check), report every mistake in every run from the same sizes ($m = 100$ at $\varepsilon = 1$), also without false reports (0 and 0); between those sizes they report less often: at $\varepsilon = 0.3$, a mistake in 100 rows in 50–60% of the runs where the program's cells report it in 80–100%, and at $\varepsilon = 3$, one in 10 rows in 40–60% against 80–90%. They spend the budget on about three times as many cells; the injected values fall in the grid's decades, so the alignment that decides the removals above does not matter here. A mistake in one row is never revealed by any private method; the run-time check stops the program at it.

10.9 Testing soundness

The theorems rest on monotonicity and soundness of the analysis, which are mechanized for the core of the language and tested for the implementation. Randomized property tests generate tables (with missing, out-of-range, pruned and dropped columns) and image folders, decide admission and deoptimization with the certificates, and compile each input afresh: every input the full certificate admits keeps every proof, and every input admitted with deoptimization compiles without new errors and keeps every removal it does not deoptimize (or stops at a re-enabled check exactly where the fresh compilation reports a violation). For the running example, 127 of 400 random tables fit the full certificate and 361 are admitted with deoptimization; 240 random image folders are admitted with deoptimization, all keeping their removals. A program with `PARSE_DATE`, `DERIVE` and a filter on the month admits 268 of 400 random batches (dates across three years, text that is not a date, divisors of 0), 243 of them deoptimizing something, none losing a removal it keeps.

What the evaluation found in our implementation. Each of the following was a case where the implementation did not follow the definitions; each is fixed and has a regression test. (1) The run-time `MATCH_RATE` check, always eliminated before certificates existed, ran for the first time on changed labels and counted broken images that `REMOVE_CORRUPT` had removed. (2) The first certificate covered only the columns the optimized program loads, so a proven `NOT_NULL` on a column pruned afterwards was unprotected. (3) Admission let Integer data into a Float column and an empty column into any type; the property test found inputs whose fresh compilation decided differently. (4) On the held-out Titanic set, scripts drop `Survived` before any use; with that `DROP` removed, admission treated the column as optional and admitted Kaggle's test file, on which the script fails. (5) The lifter translated a `drop` that pandas rejects. (6) Private compilations from false facts (RQ8) made a lifted Titanic script refuse every batch: a fill of `Embarked`, dropped right after, was removed both as a dead store and as having no effect, and the certifier kept "no missing `Embarked`" as an admission fact of a column the program never loads. A dead store is removed whatever the data, so its no-effect fact is no longer an obligation; a normal compilation of a file without missing `Embarked` had the same needless refusals waiting for the next batch. The earlier RQs were measured before this fix, which only removes admission facts.

10.10 Threats to validity

The error messages of RQ1 were chosen by the author and one person classified the issues; the mutants and the script-style interpreter are ours. Performance numbers come from one machine. The pipelines of RQ4 and RQ6 are ours, written once before running them; the months are not, and RQ5 runs other people's pipelines on the same months. Validators are used with their automatic profilers, as a new user would; a hand-tuned suite can do better, but then encodes what the certificate derives. RQ5 compares with the lifted statements, not the whole scripts, and the lifted prefixes are short. Metadata-only soundness assumes truthful Parquet statistics. The mechanization covers a core of the language; the C++ implementation is tested, not proven. The language has no joins, loops or user functions. Certificates are facts about input columns: a fact needed only after a filter is still required of every input row. Private compilation treats column names and types as public; its guarantee covers what the compiler writes, not what the data owner's runs print or how long they take (a deoptimization message names a row), which stay with the owner.

11 Related Work

Data validation. TFDV [5], Deequ [6], Great Expectations [8], pandera [7] and the Data Linter [9] check expectations on datasets when a pipeline runs, and infer them from a sample to bootstrap

Table 6: Private compilation (RQ8). Per workload, recall/precision (%) of the compilation's removals against the normal compilation of the same file: recall is the share of normal removals it also makes, precision the share of its removals that rest on true facts. Compiled: compilations without errors. Deopt. on file: compilations whose run on the compiled file itself deoptimizes (a released fact was false). Every run is scored against a fresh compilation of its batch. ε is the budget of each table of one program; "one release" serves all Titanic scripts with one budget. Generic: fixed logarithmic bins; synthetic: a table sampled from them, compiled normally.

Facts	Traffic	Bikes	Taxi	Titanic	Compilations	Compiled	Deopt. on file	Runs	Wrong
program's cells, no noise	100/100	100/100	100/100	100/100	187	187	0%	2,331	0
program's cells, $\varepsilon = 0.1$	100/100	14/100	100/100		15	15	0%	615	0
program's cells, $\varepsilon = 1$	100/100	14/100	100/100	100/17	199	199	37%	2,823	0
program's cells, $\varepsilon = 10$	100/100	100/100	100/100	100/31	199	199	16%	2,823	0
program's cells, $\varepsilon = 100$	100/100	100/100	100/100		15	15	0%	615	0
one release, $\varepsilon = 1$				100/17	368	368	40%	4,416	0
one release, $\varepsilon = 10$				100/17	368	368	40%	4,416	0
generic bins, $\varepsilon = 1$	83/100	14/100	60/100	100/17	199	199	37%	2,823	0
generic bins, $\varepsilon = 10$	71/100	86/100	60/100	100/20	199	199	30%	2,823	0
synthetic data, $\varepsilon = 1$	–/–	–/–	–/–	100/17	199	184	40%	2,208	0
synthetic data, $\varepsilon = 10$	–/–	–/–	–/–	100/20	199	184	32%	2,208	0
optimistic (no data)	29/100	14/100	40/50	100/9	187	187	46%	2,331	0
random (true or false)	26/100	30/100	56/64	44/21	398	398	13%	5,646	0
none (claims nothing)	0/–	0/–	0/–	0/–	187	187	0%	2,331	0

Table 7: Private compilation (RQ8): share of compilations that report the contract a mistake in m of 1,000,000 rows breaks (program's cells: 20 seeds, one without noise; baselines: 10 seeds). A private compilation warns that it is likely violated; one from synthetic data reports it violated. In brackets: compilations that report a contract the data does not break.

		program's cells						generic bins		synthetic data	
Mistake	m	$\varepsilon = 0.1$	$\varepsilon = 0.3$	$\varepsilon = 1$	$\varepsilon = 3$	$\varepsilon = 10$	no noise	$\varepsilon = 1$	$\varepsilon = 10$	$\varepsilon = 1$	$\varepsilon = 10$
amount above 999	0	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
amount above 999	1	0%	0%	0%	0%	0%	100%	0%	0%	0%	0%
amount above 999	10	0%	0%	0%	80%	100%	100%	0%	100%	0%	100%
amount above 999	100	0%	80%	100%	100%	100%	100%	100%	100%	100%	100%
amount above 999	1,000	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
amount above 999	10,000	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
s missing	0	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
s missing	1	0%	0%	0%	0%	0%	100%	0%	0%	0%	0%
s missing	10	0%	0%	0%	80%	100%	100%	0%	100%	0%	100%
s missing	100	0%	80%	100%	100%	100%	100%	100%	100%	100%	100%
s missing	1,000	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
s missing	10,000	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
a below 0	0	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
a below 0	1	0%	0%	0%	0%	0%	100%	0%	0%	0%	0%
a below 0	10	0%	0%	0%	90%	100%	100%	0%	100%	0%	100%
a below 0	100	0%	100%	100%	100%	100%	100%	100%	100%	100%	100%
a below 0	1,000	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
a below 0	10,000	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%

a suite. Auto-Validate [58] infers validation rules for recurring pipelines from data lakes, and Auto-Validate-by-History [59] from the statistics of past runs. All of them start from the data. A certificate starts from the program: it contains exactly the facts the compiled program's proofs and removals depend on, so it neither requires facts no proof needs (the main source of the refusals in Section 10.4) nor omits facts a removal needs (the source of the wrong runs of validators without range checks).

Static analysis of data-science code and input specifications. Urban and Müller [42] analyse which input data a program uses, and Urban [43] infers *necessary* conditions on the structure and values of the input data of Python data-processing programs by abstract interpretation; certificates are *sufficient* conditions, for the proofs of a particular compilation, found with the data at hand. Negrini et al. [44] infer the shapes of notebook data frames; Zhuang and Lu [12] type-check data-frame columns; Gerstl et al. [15] infer the shapes of R data frames and become more precise when the datasets are available; Subotić et al. [11] and Drobnjaković et al. [45] find data leakage; Lagouvardos et al. [13] infer tensor shapes; mlinspect [14] instruments pipelines. Meijer et al. [46] load a sample of the data during static analysis to check data-dependent API contracts of ML code, the closest use of data at analysis time; they report faults, whereas PrepLang compiles, removes work and carries its proofs to new data. Mirji [17] proves structural contracts of Spark pipelines at compile time and rechecks the schema at the output; Sihler et al. [16] make implicit assumptions of analysis scripts explicit for run-time checking.

Speculation and deoptimization. Dynamic compilers specialize code behind guards and deoptimize when a guard fails [34]; Flückiger et al. [51] prove such transformations correct. In data processing, Tuplex [49] compiles a fast path for the common case of a sample and sends rows that violate its assumptions to a general path, Schiavio et al. [50] speculate on input formats in Spark's SQL compiler and deoptimize at run time, and adaptive query execution switches between compiled variants [52]. These speculate for speed and fall back per row or per code region. Selective deoptimization speculates on the facts behind correctness proofs, decides once per batch on statistics the program computes anyway, re-enables exactly the removals whose certificates break, and is exact (Theorem 5.1).

Barrière et al. [61] verify speculation and deoptimization in a JIT compiler in Coq; our mechanization covers a much smaller setting. That a hint may be wrong if it is cheap to check and has a fallback is Lampson's old rule [70]; private compilation (Section 7) applies it to facts about data that are released under differential privacy, where the guards are what make noisy facts safe to optimize with.

Validity ranges and constraints that hold in the data. Progressive query optimization attaches validity ranges to the operators of a plan and re-optimizes when a cardinality leaves them [35]; parametric query optimization precomputes plans per parameter range [36]. Semantic query optimization with soft constraints mines constraints that hold in the current data and uses them to rewrite queries, dropping them when the data stops satisfying them [62]; BHUNT [63] mines fuzzy constraints, runs the rewritten query and evaluates the original one on the exception rows. These are the closest ancestors of removing work because the data allows it. Item certificates are validity ranges for correctness obligations, derived by re-analysis rather than mined or estimated, and leaving them re-enables one instruction rather than the whole plan.

Differential privacy. PrivateSQL [65], elastic sensitivity [66] and APEx [67] answer an analyst's queries over data they may not see, APEx also for data cleaning; validation servers [68] and OpenSAFELY [74] let analysts develop code on synthetic or dummy data and run it on the confidential data; PrivateClean [69] cleans locally private data. Shrinkwrap [73] and SPECIAL [75] use differentially private statistics to decide how many padding tuples secure query processing may drop, and stay exact by adding only one-sided noise, so that they never drop a real tuple. The facts a data-aware compiler needs are emptiness facts, which one-sided noise cannot certify (Section 7); a private compilation removes checks and operations on noisy facts and stays exact because selective deoptimization checks them on the data (Theorem 7.2). The questions are chosen by the program, at its own numbers, and answered with Laplace histograms [64]. Differentially private profiling collects facts about how deployed software runs [71]; languages such as Fuzz [72] prove that a program is itself differentially private. Here the program is not private; only the facts the compiler starts from are.

Residual checks and hybrid checking. Soft typing [20] and hybrid type checking [19] discharge what they can statically and check the rest at run time; assertion-checking systems reduce run-time checks by abstract interpretation [53], and sound gradual verification inserts checks for weakened specifications [54]. PrepLang does this per batch: a check is residual only for batches that break its certificate.

Preconditions, abduction and certificates. Necessary preconditions [37], and sufficient ones found by backward under-approximation [48], are conditions on a program's inputs for its assertions; abductive inference [47] finds the weakest facts that discharge a proof obligation. An item certificate is such a sufficient condition for one removal, restricted to a finite lattice anchored at the program's constants, so that a forward analyzer used as a black box finds a maximal one; DeepInfer [38] infers data preconditions of trained networks. Proof-carrying code [33] and abstraction-carrying code [55] ship a proof or an abstract model that a consumer checks; a data certificate ships the facts the proofs need, and the generated program checks every new input against them.

Contracts and composition. Assume-guarantee contracts compose components by refinement [56]; schema registries check that a reader schema can read what a writer schema produces [57]; data contracts between producers and consumers are written by hand. In PrepLang both sides of the containment are derived from the stages' proofs, and the producer contract is synthesized from the consumers' code.

Types from data and data-centric compilers. F# type providers [18] infer types from sample data at compile time. Min/max statistics per block let databases skip data [24]; PrepLang uses the same Parquet statistics [26] to prove contracts. Halide [27], Weld [28], Dias [40] and Magpie [41] optimize data code driven by the program; PrepLang's optimizations are driven by facts about the data, and its certificates say when they still apply.

12 Conclusion

A data-aware compiler proves facts about one batch; data certificates say which of them the compiled program depends on, selective deoptimization keeps the compiled program usable for batches that break some of them, and composition carries guarantees from one stage to the next. On real monthly data, the compiled program handles 93% of new months exactly as a fresh compilation would, without recompiling, where validators set up on the same data refuse most months or let wrong results through; on real scripts, including pipelines written by others and run on the real months, it reproduces the scripts' results on every batch it runs. Because deoptimization never relies on the compile-time facts being true, the compiler can also work from facts it may not trust: compiled from differentially private answers, or from random ones, every run stayed exact, and an analyst who may not see the data still learns which contracts it breaks once enough rows break them. The refusals that remain come from facts no run-time code can replace, such as a publisher changing a column's type, and point to the next step: certificates that let generated code adapt to such changes, joins in the language, and the mechanized soundness proof extended from the core to the whole analysis.

Artifact. The compiler, the Lean development, the pandas front end, the composition tool, the evaluation scripts with their pinned

baseline environments, and the raw results are kept in one repository; `eval/paper_numbers.py` regenerates every number, table and data figure of this paper from the results.

References

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.

[2] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data lifecycle challenges in production machine learning: A survey," *ACM SIGMOD Record*, vol. 47, no. 2, pp. 17–28, 2018.

[3] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.

[4] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[5] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. on Machine Learning and Systems (MLSys)*, 2019.

[6] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[7] N. Bantilan, "pandera: Statistical data validation of pandas dataframes," in *Proc. 19th Python in Science Conf. (SciPy)*, 2020, pp. 116–124.

[8] Great Expectations, "Great Expectations: data quality testing," https://greatexpectations.io.

[9] N. Hynes, D. Sculley, and M. Terry, "The Data Linter: Lightweight, automated sanity checking for ML data sets," in *NIPS Workshop on Machine Learning Systems*, 2017.

[10] C. Urban, "Static analysis of data science software," in *Static Analysis Symposium (SAS)*, LNCS 11822, 2019, pp. 17–23.

[11] P. Subotić, L. Milikić, and M. Stojić, "A static analysis framework for data science notebooks," in *Proc. 44th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2022, pp. 13–22.

[12] Y. Y. Zhuang and M.-Y. Lu, "Enabling type checking on columns in data frame libraries by abstract interpretation," *IEEE Access*, vol. 10, 2022.

[13] S. Lagouvardos, J. Dolby, N. Grech, A. Antoniadis, and Y. Smaragdakis, "Static analysis of shape in TensorFlow programs," in *Proc. 34th European Conf. on Object-Oriented Programming (ECOOP)*, LIPIcs vol. 166, 2020, pp. 15:1–15:29.

[14] S. Grafberger, P. Groth, J. Stoyanovich, and S. Schelter, "Data distribution debugging in machine learning pipelines," *The VLDB Journal*, vol. 31, pp. 1103–1126, 2022.

[15] O. Gerstl, F. Sihler, and M. Tichy, "Inferring the shape of data frames in R programs using abstract interpretation," arXiv:2607.03889, 2026.

[16] F. Sihler, L. Pfrenger, O. Gerstl, and M. Tichy, "Towards automatically inferring constraints to identify implicit assumptions in data analysis," arXiv:2607.03114, 2026.

[17] V. Mirji, "Shift schema drift left: Policy-aware compile-time contracts for typed JVM and Spark pipelines," arXiv:2604.16986, 2026.

[18] T. Petricek, G. Guerra, and D. Syme, "Types from data: Making structured data first-class citizens in F#," in *Proc. 37th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2016, pp. 477–490.

[19] C. Flanagan, "Hybrid type checking," in *Proc. 33rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, 2006, pp. 245–256.

[20] R. Cartwright and M. Fagan, "Soft typing," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 1991, pp. 278–292.

[21] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th ACM Symp. on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.

[22] P. Cousot and R. Cousot, "Static determination of dynamic properties of programs," in *Proc. 2nd Int. Symp. on Programming*, Paris, 1976, pp. 106–130.

[23] E. F. Codd, "Extending the database relational model to capture more meaning," *ACM Trans. Database Systems*, vol. 4, no. 4, pp. 397–434, 1979.

[24] G. Moerkotte, "Small materialized aggregates: A light weight index structure for data warehousing," in *Proc. 24th Int. Conf. on Very Large Data Bases (VLDB)*, 1998, pp. 476–487.

[25] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, and T. Vassilakis, "Dremel: Interactive analysis of web-scale datasets," *Proc. VLDB Endowment*, vol. 3, no. 1–2, pp. 330–339, 2010.

[26] Apache Parquet, "File format," https://parquet.apache.org/docs/file-format/.

[27] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *Proc. 34th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2013, pp. 519–530.

[28] S. Palkar, J. J. Thomas, A. Shanbhag, D. Narayanan, H. Pirk, M. Schwarzkopf, S. Amarasinghe, and M. Zaharia, "Weld: A common runtime for high performance data analytics," in *Proc. Conf. on Innovative Data Systems Research (CIDR)*, 2017.

[29] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Trans. Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[30] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, "A comprehensive study on deep learning bug characteristics," in *Proc. 27th ACM Joint Meeting on European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, 2019, pp. 510–520.

[31] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *Proc. 42nd Int. Conf. on Software Engineering (ICSE)*, 2020, pp. 1110–1121.

[32] Q. Huynh-Thu and M. Ghanbari, "Scope of validity of PSNR in image/video quality assessment," *Electronics Letters*, vol. 44, no. 13, pp. 800–801, 2008.

[33] G. C. Necula, "Proof-carrying code," in *Proc. 24th ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, 1997, pp. 106–119.

[34] U. Hölzle, C. Chambers, and D. Ungar, "Debugging optimized code with dynamic deoptimization," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 1992, pp. 32–43.

[35] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdzic, "Robust query processing through progressive optimization," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2004, pp. 659–670.

[36] Y. E. Ioannidis, R. T. Ng, K. Shim, and T. K. Sellis, "Parametric query optimization," in *Proc. 18th Int. Conf. on Very Large Data Bases (VLDB)*, 1992, pp. 103–114.

[37] P. Cousot, R. Cousot, M. Fähndrich, and F. Logozzo, "Automatic inference of necessary preconditions," in *Proc. 14th Int. Conf. on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, LNCS 7737, 2013, pp. 128–148.

[38] S. Ahmed, H. Gao, and H. Rajan, "Inferring data preconditions from deep learning models for trustworthy prediction in deployment," in *Proc. 46th Int. Conf. on Software Engineering (ICSE)*, 2024.

[39] W. D. Clinger, "How to read floating point numbers accurately," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 1990, pp. 92–101.

[40] S. Baziotis, D. Kang, and C. Mendis, "Dias: Dynamic rewriting of pandas code," *Proc. ACM on Management of Data (SIGMOD)*, vol. 2, no. 1, 2024.

[41] A. Jindal, K. V. Emani, M. Daum, O. Poppe, B. Haynes, A. Pavlenko, A. Gupta, K. Ramachandra, C. Curino, A. Mueller, W. Wu, and H. Patel, "Magpie: Python at speed and scale using cloud backends," in *Proc. Conf. on Innovative Data Systems Research (CIDR)*, 2021.

[42] C. Urban and P. Müller, "An abstract interpretation framework for input data usage," in *Proc. 27th European Symp. on Programming (ESOP)*, LNCS 10801, 2018, pp. 683–710.

[43] C. Urban, "What programs want: Automatic inference of input data specifications," arXiv:2007.10688, 2020.

[44] L. Negrini, G. Shabadi, and C. Urban, "Static analysis of data transformations in Jupyter notebooks," in *Proc. 12th ACM SIGPLAN Int. Workshop on the State Of the Art in Program Analysis (SOAP)*, 2023.

[45] F. Drobnjaković, P. Subotić, and C. Urban, "Abstract interpretation-based data leakage static analysis," arXiv:2211.16073, 2022.

[46] W. Meijer, K. Sandahl, and D. Varró, "Data-aware static analysis: Improving detection of semantic faults in machine learning code using data characteristics," in *Proc. 48th Int. Conf. on Software Engineering, New Ideas and Emerging Results (ICSE-NIER)*, 2026. arXiv:2606.09957.

[47] I. Dillig, T. Dillig, and A. Aiken, "Automated error diagnosis using abductive inference," in *Proc. 33rd ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2012, pp. 181–192.

[48] A. Miné, "Backward under-approximations in numeric abstract domains to automatically infer sufficient program conditions," *Science of Computer Programming*, vol. 93, pp. 154–182, 2014.

[49] L. Spiegelberg, R. Yesantharao, M. Schwarzkopf, and T. Kraska, "Tuplex: Data science in Python at native code speed," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2021, pp. 1718–1731.

[50] F. Schiavio, D. Bonetta, and W. Binder, "Dynamic speculative optimizations for SQL compilation in Apache Spark," *Proc. VLDB Endowment*, vol. 13, no. 5, pp. 754–767, 2020.

[51] O. Flückiger, G. Scherer, M.-H. Yee, A. Goel, A. Ahmed, and J. Vitek, "Correctness of speculative optimizations with dynamic deoptimization," *Proc. ACM on Programming Languages*, vol. 2, no. POPL, 2018.

[52] A. Kohn, V. Leis, and T. Neumann, "Adaptive execution of compiled queries," in *Proc. 34th IEEE Int. Conf. on Data Engineering (ICDE)*, 2018, pp. 197–208.

[53] N. Stulova, J. F. Morales, and M. V. Hermenegildo, "Some trade-offs in reducing the overhead of assertion run-time checks via static analysis," *Science of Computer Programming*, vol. 155, pp. 3–26, 2018.

[54] C. Zimmerman, J. DiVincenzo, and J. Aldrich, "Sound gradual verification with symbolic execution," *Proc. ACM on Programming Languages*, vol. 8, no. POPL, 2024.

[55] E. Albert, G. Puebla, and M. Hermenegildo, "Abstraction-carrying code," in *Proc. 11th Int. Conf. on Logic for Programming, Artificial Intelligence and Reasoning (LPAR)*, LNCS 3452, 2004, pp. 380–397.

[56] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, and K. G. Larsen, "Contracts for system design," *Foundations and Trends in Electronic Design Automation*, vol. 12, no. 2–3, pp. 124–400, 2018.

[57] The Apache Software Foundation, "Apache Avro specification: Schema resolution," https://avro.apache.org/docs/1.12.0/specification/, 2024.

[58] J. Song and Y. He, "Auto-Validate: Unsupervised data validation using data-domain patterns inferred from data lakes," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2021, pp. 1678–1691.

[59] D. Tu, Y. He, W. Cui, S. Ge, H. Zhang, H. Shi, D. Zhang, and S. Chaudhuri, "Auto-Validate by-History: Auto-program data quality constraints to validate recurring data pipelines," in *Proc. 29th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD)*, 2023.

[60] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th Int. Conf. on Automated Deduction (CADE)*, LNCS 12699, 2021, pp. 625–635.

[61] A. Barrière, S. Blazy, O. Flückiger, D. Pichardie, and J. Vitek, "Formally verified speculation and deoptimization in a JIT compiler," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, pp. 1–26, 2021.

[62] P. Godfrey, J. Gryz, and C. Zuzarte, "Exploiting constraint-like data characterizations in query optimization," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2001, pp. 582–592.

[63] P. G. Brown and P. J. Haas, "BHUNT: Automatic discovery of fuzzy algebraic constraints in relational data," in *Proc. 29th Int. Conf. on Very Large Data Bases (VLDB)*, 2003, pp. 668–679.

[64] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[65] I. Kotsogiannis, Y. Tao, X. He, M. Fanaeian, A. Machanavajjhala, M. Hay, and G. Miklau, "PrivateSQL: A differentially private SQL query engine," *Proc. VLDB Endowment*, vol. 12, no. 11, pp. 1371–1384, 2019.

[66] N. Johnson, J. P. Near, and D. Song, "Towards practical differential privacy for SQL queries," *Proc. VLDB Endowment*, vol. 11, no. 5, pp. 526–539, 2018.

[67] C. Ge, X. He, I. F. Ilyas, and A. Machanavajjhala, "APEx: Accuracy-aware differentially private data exploration," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2019, pp. 177–194.

[68] A. F. Barrientos, A. Bolton, T. Balmat, J. P. Reiter, J. M. de Figueiredo, A. Machanavajjhala, Y. Chen, C. Kneifel, and M. DeLong, "Providing access to confidential research data through synthesis and verification: An application to data on employees of the U.S. federal government," *Annals of Applied Statistics*, vol. 12, no. 2, pp. 1124–1156, 2018.

[69] S. Krishnan, J. Wang, M. J. Franklin, K. Goldberg, and T. Kraska, "PrivateClean: Data cleaning and differential privacy," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2016, pp. 937–951.

[70] B. W. Lampson, "Hints for computer system design," in *Proc. 9th ACM Symp. on Operating Systems Principles (SOSP)*, 1983, pp. 33–48.

[71] H. Zhang, Y. Hao, S. Latif, R. Bassily, and A. Rountev, "Differentially-private software frequency profiling under linear constraints," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, pp. 203:1–203:24, 2020.

[72] J. Reed and B. C. Pierce, "Distance makes the types grow stronger: A calculus for differential privacy," in *Proc. 15th ACM SIGPLAN Int. Conf. on Functional Programming (ICFP)*, 2010, pp. 157–168.

[73] J. Bater, X. He, W. Ehrich, A. Machanavajjhala, and J. Rogers, "Shrinkwrap: Efficient SQL query processing in differentially private data federations," *Proc. VLDB Endowment*, vol. 12, no. 3, pp. 307–320, 2018.

[74] E. J. Williamson, A. J. Walker, K. Bhaskaran, *et al.*, "Factors associated with COVID-19-related death using OpenSAFELY," *Nature*, vol. 584, pp. 430–436, 2020.

[75] C. Wang, L. Qiu, J. Bater, and Y. Luo, "SPECIAL: Synopsis assisted secure collaborative analytics," *Proc. VLDB Endowment*, vol. 18, no. 4, pp. 1035–1048, 2024.